EXPERIMENT-1 (question 1)

Name: Simran Jangid

Roll_no: 2520030166

Section: 8

Experiment No. 1 (question 1)

Title: Installation of Linux VM, GCC Configuration, Git Repository Setup, Shell Architecture Analysis, and Process Creation using `fork()`, `exec()`, and System Call Tracing.

Aim: To set up a Linux development environment (VM, GCC, Git, Makefile, Project Structure) and write a C program to demonstrate process abstraction, execution of the exec() family, parent-child relationships, process tree inspection, and system call tracing using strace**.**

Objective:

- To set up a functional Linux environment, build system (`make`), and revision control (`git`).
- To understand shell architecture and process abstraction in operating systems.
- To learn process creation using the `fork()` system call.
- To execute external binaries using the `exec()` family of system calls (`execlp`, `execvp`).
- To observe process relationships (parent-child, process trees using `pstree`/`ps`).
- To trace and analyze system calls made by a process using tools like `strace`.

- **Theory:**

- **Linux Development Environment & Shell Architecture:**

- **GCC & Makefile:** GCC (GNU Compiler Collection) compiles C programs into executable binaries. A `Makefile` automates the compilation process using rules and target dependencies.
- **Git:** A distributed version control system used to track changes in code files and manage project structure.
- **Shell Architecture:** The Linux shell acts as a command-line interface (CLI) that reads user input, parses commands, creates child processes using `fork()`, replaces process images using `exec()`, and waits for completion using `wait()`.

- **Process Abstraction & Management System Calls:**

1. fork():

- Creates a new child process.
- The child process is an exact copy of the parent.
- Returns:
    - 0 to the child process.
    - Child PID to the parent process.
    - -1 if creation fails.

2. exec():

The exec() family replaces the current process image with a new program.

Common functions are:

- execl()
- execv()
- execlp()
- execvp()

In this experiment, the child process executes the user-entered Linux command.

3. wait():

The parent process waits until the child process finishes execution.

This prevents zombie processes.

Algorithm:

1. Start the program.
2. Accept a Linux command from the user.
3. Create a child process using fork().
4. If child process:
    - Execute the command using execlp() or execvp().
5. If parent process:
    - Wait for the child process using wait().
6. Display Parent PID and Child PID.

7. Stop.

System Calls Used:

Command / Call	Purpose
`fork()`	Creates a new child process.
`execvp()` / `execlp()`	Replaces current process with a target command.
`wait()`	Suspends parent execution until child terminates.
`getpid()` / `getppid()`	Returns current PID / parent PID.
`strace`	Intercepts and logs system calls executed by a program.
`pstree`	Displays process hierarchy as a tree diagram.
`make`	Automates compilation based on `Makefile` instructions.

Sample Output:

Passing Students: {'Alice': 85, 'Charlie': 92, 'Evan': 78}

Average Score: 85.00

- **Algorithm:**

- Initialize project directory structure and initialize a Git repository using `git init`.
- Create a `Makefile` to compile source code with `gcc`.
- Read the user input (command name).
- Call `fork()` to spawn a child process.
- **Child Process (`PID == 0`):**
- Display child PID and parent PID.
- Execute command using `execlp()` / `execvp()`.
- **Parent Process (`PID > 0`):**
- Display parent PID and created child PID.
- Call `wait(NULL)` to block until the child finishes.
- Run the executable with `strace` to observe low-level system call sequences (`clone`/`fork`, `execve`, `wait4`).

Result:

The program was successfully executed. The Linux command entered by the user was executed through a child process. The parent process waited for the child process to complete execution, and the Process IDs of both parent and child were displayed successfully.

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
  pid_t pid;

  printf("Parent process started (PID: %d)\n", getpid());

  pid = fork();  // create child process

  if (pid < 0) {
    perror("Fork failed");
    return 1;
  }
  else if (pid == 0) {
    // Child process
    printf("Child process (PID: %d, Parent PID: %d)\n", getpid(), getppid());

    // Replace child with "ls" program using exec
    execlp("ls", "ls", "-l", NULL);
```

```
    // If exec fails
    perror("Exec failed");
  }
  else {
    // Parent process waits for child
    wait(NULL);
    printf("Parent process (PID: %d) finished waiting for child.\n", getpid());
  }

  return 0;
}
```

OUPUT: